Prueba técnica — Desarrollador TI/Sénior

Seguros Bolívar · Candidato: **Daniel León** · danielarevalodev2005@gmail.com

Este documento único reúne las respuestas de los cinco módulos de la prueba.

Código del Módulo 2 (repositorio): https://github.com/DaniArevalo12/Daniel_Leon_Prueba_Tecnica

Contenido

Módulo	Contenido	Formato
1. Diseño de sistema	Arquitectura, patrones, modelo de datos y diagramas de componentes	Texto y diagramas
2. Prueba técnica práctica	API de pólizas (Spring Boot + Angular). El código está en el repositorio de GitHub	Resumen en este documento y código en GitHub
3. Base de datos	Estrategias de optimización SQL	Texto
4. Versionamiento	Comando y justificación (Git)	Texto
5. Liderazgo técnico	Respuestas a las preguntas de gestión y liderazgo	Texto

Módulo 1: Diseño de sistema

Módulo 1: Diseño de sistema · Prueba técnica Seguros Bolívar · Versión 1.0

> Este documento describe la arquitectura objetivo para producción. La API construida en el Módulo 2 (`02-api-polizas/`) implementa una parte de ella como monolito modular. La sección 8 indica con exactitud qué está implementado y qué es propuesta.

1. Resumen ejecutivo y supuestos

La plataforma gestiona pólizas de arrendamiento **individuales** (un riesgo, el arrendatario es tomador y asegurado) y **colectivas** (inmobiliarias y copropiedades, de 1 a n riesgos), con prima = canon × meses y **renovación automática** ajustada por IPC.

Se propone:

- **Tres microservicios de negocio** (Pólizas, Riesgos y Notificaciones) detrás de un **API Gateway**.
- Un **Adapter CORE** que actúa como capa anticorrupción frente al servicio de edición de **WebLogic**.
- Integración **orientada a eventos con Kafka y Transactional Outbox**: ningún cambio se pierde, aunque el CORE legado esté caído.

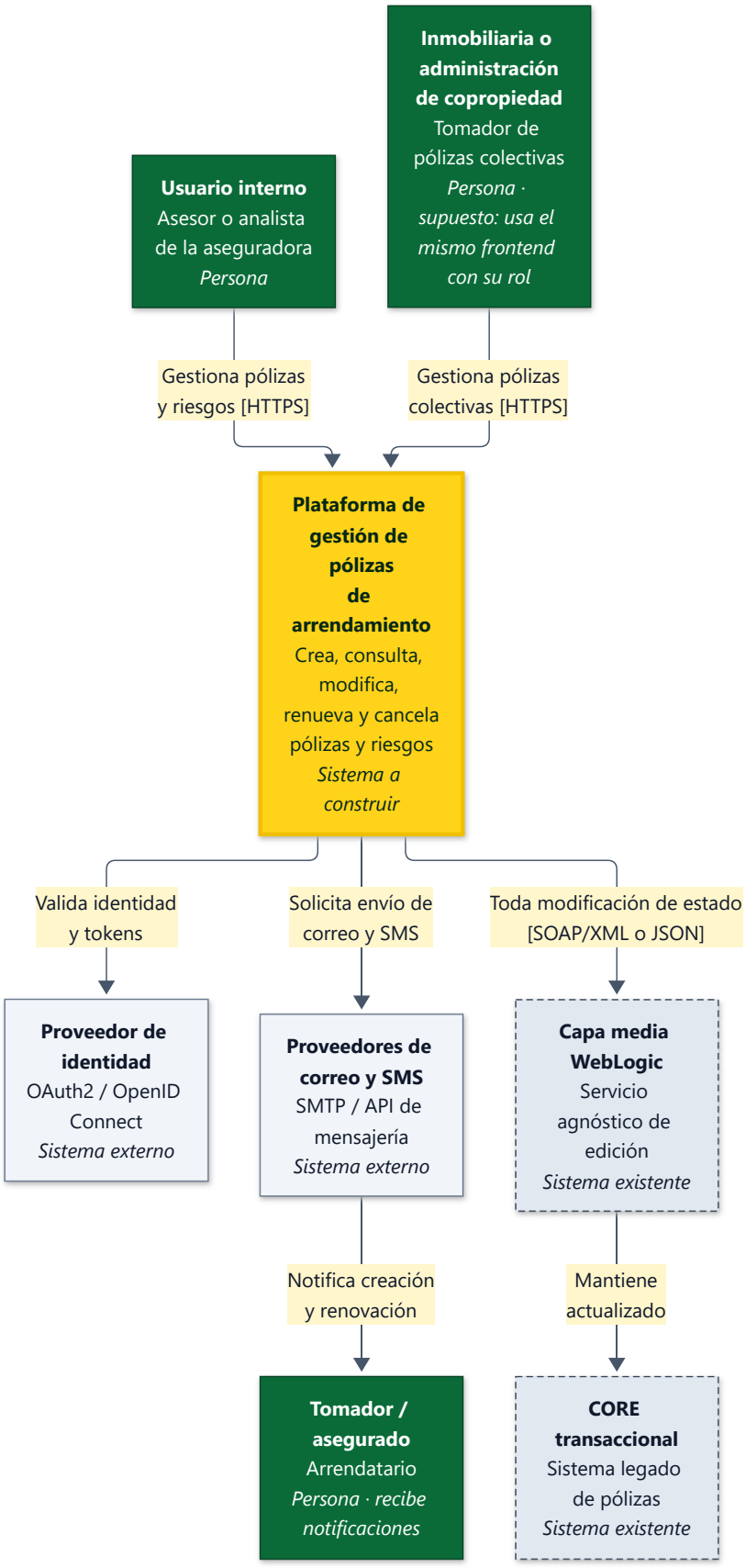
La plataforma es la fuente de verdad operativa. El CORE se sincroniza de forma asíncrona, idempotente y observable, y cada póliza y riesgo expone su **estado de sincronización**. Con despliegue multi-zona y Oracle con Data Guard, el objetivo de disponibilidad es **99,9 %**.

Supuestos. No hay información de la empresa para validarlos:

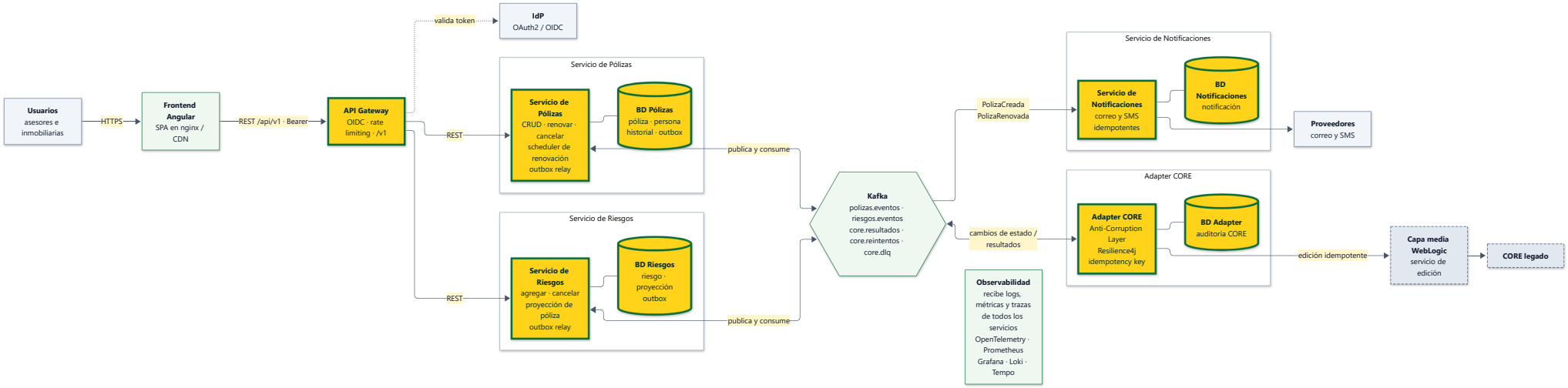
#	Supuesto	Impacto si cambia
S1	No se conocen volúmenes ni picos; se diseña para escalar horizontalmente	Dimensionamiento de réplicas y particiones
S2	El contrato de WebLogic (SOAP/XML o JSON) y sus códigos de error son desconocidos	Solo cambia el Adapter CORE (ACL)
S3	No se sabe si WebLogic acepta una clave de idempotencia	Si no la acepta, el Adapter deduplica con su tabla de auditoría (§3.3)
S4	Existe un proveedor de identidad corporativo compatible con OIDC	Gateway y frontend
S5	Oracle es la base de datos corporativa	Modelo y alta disponibilidad (Data Guard)
S6	Correo y SMS se envían mediante proveedores externos	Servicio de Notificaciones
S7	El IPC lo parametriza negocio (fuente oficial: DANE) y la renovación automática corre N días antes del vencimiento (N configurable)	Scheduler de renovación
S8	Las inmobiliarias usan el mismo frontend con un rol propio	Contexto y autorización

2. Arquitectura de alto nivel

2.1 Contexto (C4 nivel 1)



2.2 Contenedores (C4 nivel 2)



En amarillo con borde grueso están los seis componentes que pide el enunciado: API Gateway, Servicio de Pólizas, Servicio de Riesgos, Servicio de Notificaciones, Adapter CORE y bases de datos.

2.3 Responsabilidades

Contenedor	Responsabilidad	Datos propios	Escala por
API Gateway	Autenticación OIDC, autorización por *scopes*, rate limiting, enrutamiento y versionamiento `/v1`	—	Peticiones por segundo
Servicio de Pólizas	Crear, consultar, modificar, renovar (manual y automática) y cancelar pólizas; scheduler de renovación con ShedLock; publicar eventos de póliza	BD Pólizas	Peticiones; un solo nodo ejecuta el job a la vez
Servicio de Riesgos	Agregar y cancelar riesgos validando contra una **proyección local** de la póliza; cancelar en cascada al recibir `PolizaCancelada`	BD Riesgos	Peticiones y particiones de Kafka
Servicio de Notificaciones	Correo y SMS en creación y renovación, idempotentes por evento y canal	BD Notificaciones	Particiones de Kafka
Adapter CORE	Anticorrupción frente a WebLogic: traducción, idempotency key, reintentos, circuit breaker, DLQ y auditoría	BD Adapter (auditoría)	Particiones; limitado por la capacidad de WebLogic (bulkhead)
Kafka	Transporte de eventos con orden por póliza (clave de partición = `polizaId`)	Topics	Brokers y particiones
Bases de datos	Oracle con un esquema por servicio; ningún servicio lee el esquema de otro	—	Réplicas de lectura (Active Data Guard)

3. Patrones de arquitectura

3.1 Microservicios + API Gateway

Problema en este caso	Las capacidades tienen ritmos distintos: las consultas de pólizas reciben mucho tráfico, la renovación es un proceso batch diario, las notificaciones dependen de proveedores externos y la integración con el CORE está limitada por un sistema legado. En un único despliegue, un pico o una falla en una arrastra a las demás.
Cómo se aplica	Pólizas, Riesgos y Notificaciones se despliegan y escalan por separado, cada uno con su esquema. El Gateway es la única entrada: valida tokens OIDC, aplica rate limiting y expone `/api/v1`.
Trade-offs	Más piezas que operar (CI/CD, observabilidad distribuida). La póliza deja de estar en la misma transacción que sus riesgos, lo que introduce **consistencia eventual** (ver abajo).
Alternativa descartada	Monolito único: más simple, pero escala y falla como una sola unidad.

Separación de Pólizas y Riesgos: consistencia eventual con proyección local. En el dominio, la póliza es la raíz del agregado de sus riesgos. Al separarlos:

- **Riesgos mantiene** `POLIZA_PROYECCION` (tipo, estado, versión), alimentada por eventos de póliza. Con ella valida, sin llamar a Pólizas, las reglas "solo colectivas" (422) y "no canceladas" (409).
- **La cascada de cancelación** es el evento `PolizaCancelada`, que Riesgos consume de forma idempotente y con el que cancela todos los riesgos activos de esa póliza.
- **Ventana de inconsistencia.** Si alguien agrega un riesgo justo cuando la póliza se cancela, el sistema converge: el consumidor cancela todo riesgo activo, incluido el recién creado. La ventana dura lo que tarda en propagarse el evento (normalmente milisegundos).
- **Proyección ausente.** Si la póliza aún no llegó a la proyección, Riesgos consulta a Pólizas de forma síncrona (*read-through*) en lugar de rechazar.

Alternativa válida y más fiel al DDD: un solo servicio "Pólizas y Riesgos" con Riesgos como módulo interno. Conserva el agregado en una transacción (sin proyección ni ventana de inconsistencia) y la preferiría con un **equipo pequeño, bajo volumen o poca diferencia de carga** entre ambas capacidades. Es el **punto de partida natural**: el Módulo 2 ya es ese monolito modular, con las reglas en el agregado `Poliza` y los puertos definidos. Extraer Riesgos después es un cambio de despliegue, no de modelo.

3.2 Arquitectura hexagonal + Anti-Corruption Layer (ACL)

Problema en este caso	El CORE es legado y su contrato (SOAP/XML o JSON vía WebLogic) no debe contaminar el modelo de dominio. Además, es la integración con más riesgo de falla.
Cómo se aplica	El dominio declara un **puerto** (`CorePort`) con su propio lenguaje: `EventoCore(evento, operacion, polizaId)` . El **Adapter CORE** lo traduce al contrato de WebLogic, agrega la idempotency key, aplica Resilience4j y audita cada llamada. Los servicios de negocio no conocen WebLogic.
Trade-offs	Una capa de traducción más que mantener. Los cambios en el contrato del CORE pasan por el adapter.
Alternativa descartada	Llamar a WebLogic desde los servicios de negocio: acopla el dominio a un contrato legado y dispersa la lógica de reintentos.

Ya implementado en el Módulo 2: `CorePort` (dominio) + `CoreHttpAdapter` (infraestructura) con timeouts. Hoy la llamada es síncrona y un fallo solo se registra en logs; la evolución es mover ese adaptador a un servicio que consume del outbox.

3.3 Event-driven + Transactional Outbox

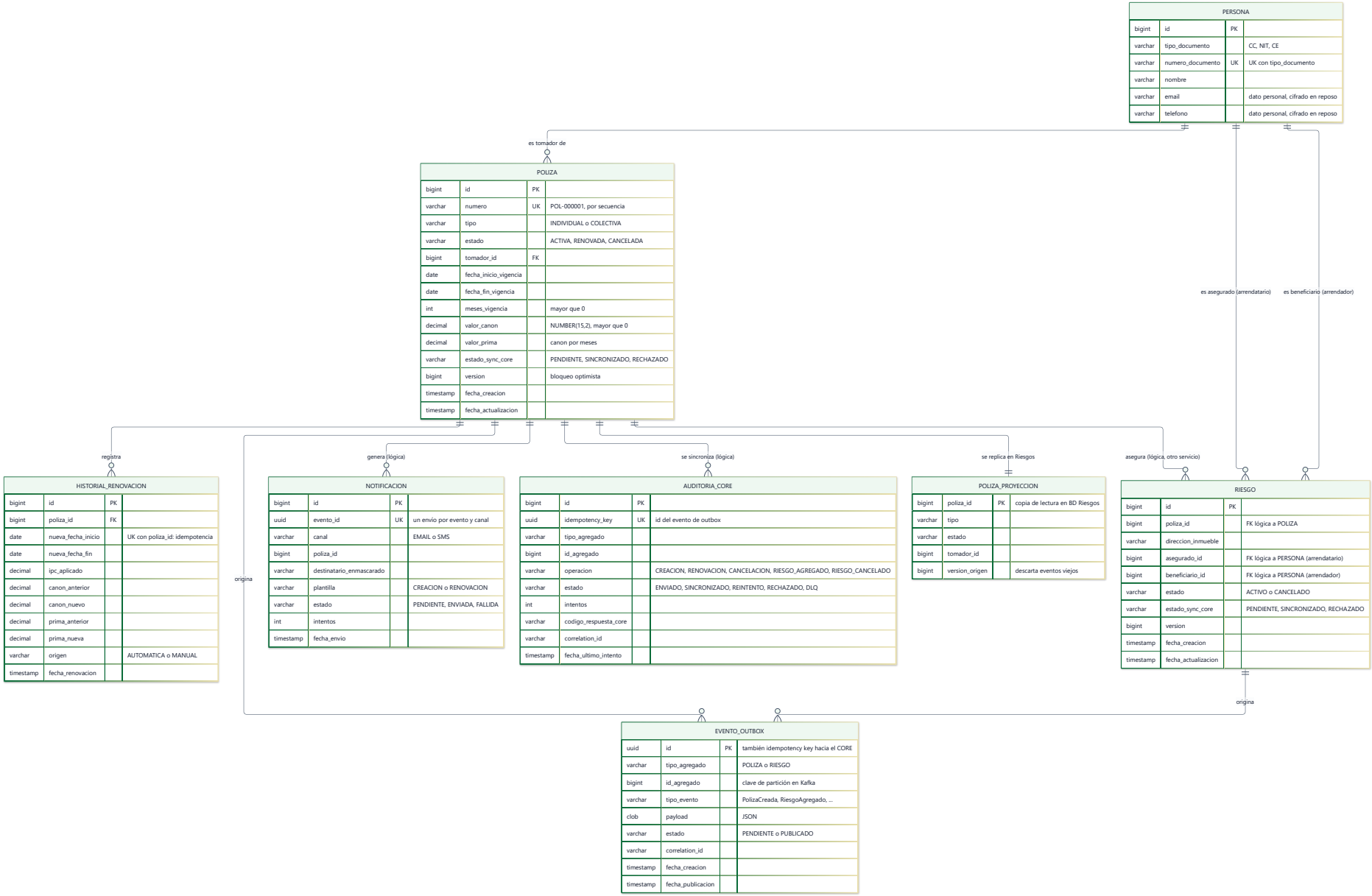
Problema en este caso	Al crear o renovar hay que notificar y, en **toda** modificación, avisar al CORE. Escribir en la base de datos y publicar en Kafka son dos sistemas: si uno falla entre ambos pasos (*dual write*), se pierden eventos o se publican cambios que nunca se confirmaron.
Cómo se aplica	En la **misma transacción** del cambio se inserta una fila en `EVENTO_OUTBOX` . Un *relay* (consulta con `SELECT … FOR UPDATE SKIP LOCKED` , o CDC con Debezium) publica en Kafka con clave `polizaId` y la marca como publicada. Notificaciones y el Adapter CORE consumen. El `id` del evento es la **idempotency key** hacia WebLogic y hacia los consumidores.
Trade-offs	Entrega **al menos una vez**: los consumidores deben ser idempotentes. Hay latencia entre el cambio y su efecto en el CORE, y el outbox se debe purgar (particionado por fecha).
Alternativa descartada	Eventos en memoria tras el commit (lo que hace hoy el Módulo 2 con `@TransactionalEventListener`): si el proceso cae justo después del commit, el evento se pierde.

Idempotencia hacia el CORE: cada llamada a WebLogic lleva `Idempotency-Key = id del evento` . Los reintentos repiten la misma clave, así que el CORE no duplica ediciones. Si WebLogic no soporta claves (S3), el Adapter consulta `AUDITORIA_CORE` (clave única) antes de reenviar y, si existe, la operación de consulta del CORE.

3.4 CQRS: evaluado y diferido

Las lecturas son consultas simples por filtro e identificador, y no hay evidencia de una carga de lectura muy asimétrica (S1). CQRS completo (modelos y almacenes de lectura separados) agregaría otra fuente de consistencia eventual sin beneficio demostrado. **Se difiere.** Si las consultas crecen, el primer paso son **réplicas de lectura** (Active Data Guard) y caché; CQRS se reconsidera si aparecen vistas de reporte complejas.

4. Modelo de datos



Reglas que impone el modelo

Regla	Cómo se garantiza
INDIVIDUAL tiene exactamente 1 riesgo y el arrendatario es el tomador	Validación del agregado al crear (`riesgo.asegurado_id = poliza.tomador_id`) y en Riesgos con la proyección
COLECTIVA tiene de 1 a n riesgos; arrendatario = asegurado y arrendador = beneficiario	`asegurado_id` y `beneficiario_id` de cada riesgo apuntan a `PERSONA`
prima = canon × meses, con dinero en `NUMBER(15,2)`	Calculada en el dominio (escala 2, `HALF_EVEN`) más `CHECK` en BD
Una renovación por vigencia (idempotencia del job)	`UNIQUE (poliza_id, nueva_fecha_inicio)` en `HISTORIAL_RENOVACION`
Una notificación por evento y canal	`UNIQUE (evento_id, canal)` en `NOTIFICACION`
Una edición en el CORE por evento	`UNIQUE (idempotency_key)` en `AUDITORIA_CORE`
Sin actualizaciones perdidas por concurrencia	Columna `version` (bloqueo optimista) que responde 409
El estado del CORE es visible	`estado_sync_core` ∈ {PENDIENTE, SINCRONIZADO, RECHAZADO} en `POLIZA` y `RIESGO`

Base de datos por servicio. Son esquemas Oracle separados en el mismo clúster con alta disponibilidad: aislamiento lógico sin multiplicar licencias ni operación.

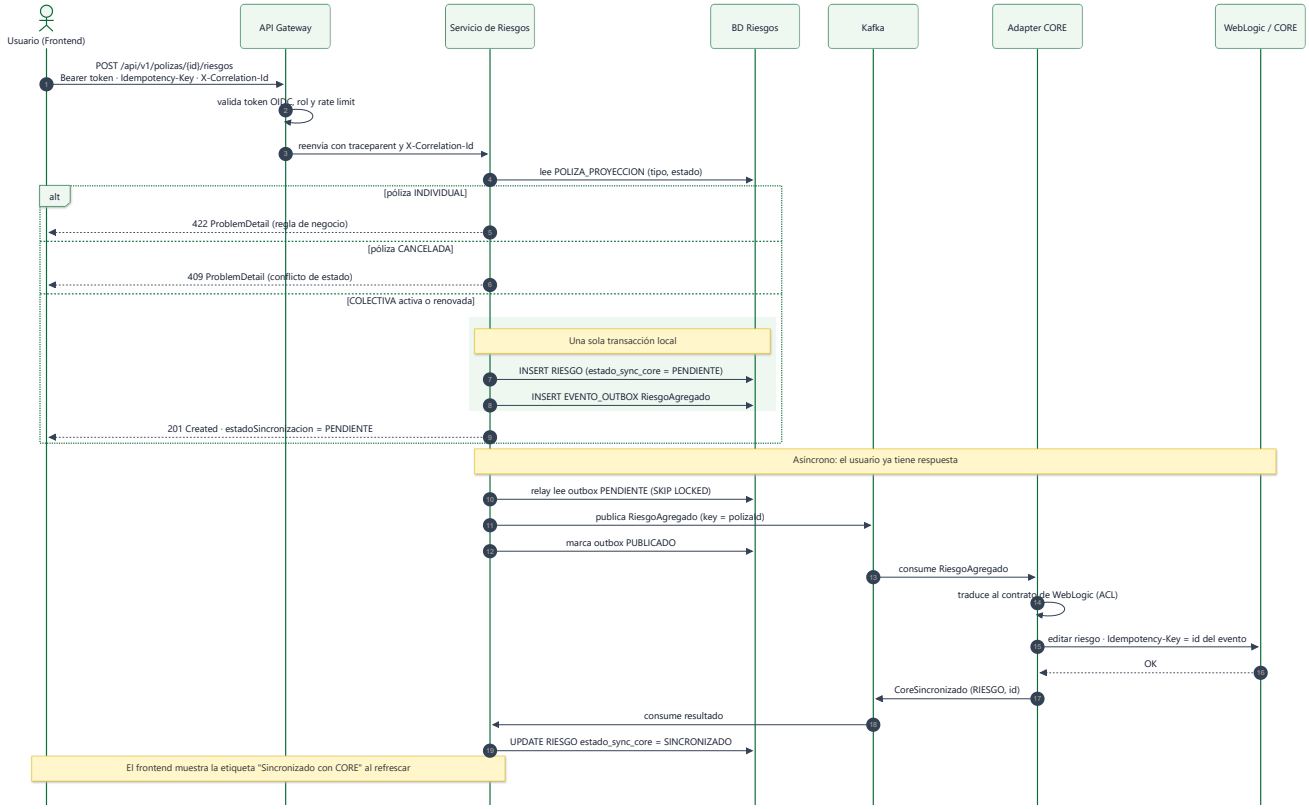
Servicio	Esquema	Tablas
Pólizas	`POLIZAS`	`PERSONA` , `POLIZA` , `HISTORIAL_RENOVACION` , `EVENTO_OUTBOX` , `SHEDLOCK`
Riesgos	`RIESGOS`	`RIESGO` , `POLIZA_PROYECCION` , `EVENTO_OUTBOX`
Notificaciones	`NOTIFICACIONES`	`NOTIFICACION`
Adapter CORE	`CORE_ADAPTER`	`AUDITORIA_CORE`

Entre esquemas solo hay **FK lógicas** (sin `FOREIGN KEY` física): cada servicio puede evolucionar y desplegarse por separado. `PERSONA` vive en Pólizas mientras no exista un maestro de clientes corporativo; si existe, se reemplaza por una referencia a él.

Diferencia con el Módulo 2: allí tomador, arrendatario y arrendador son texto, y no hay `PERSONA` , historial, outbox ni `estado_sync_core` (decisión documentada en `02-api-polizas/backend/DECISIONES.md`).

5. Flujos clave

5.1 Agregar riesgo a una póliza colectiva

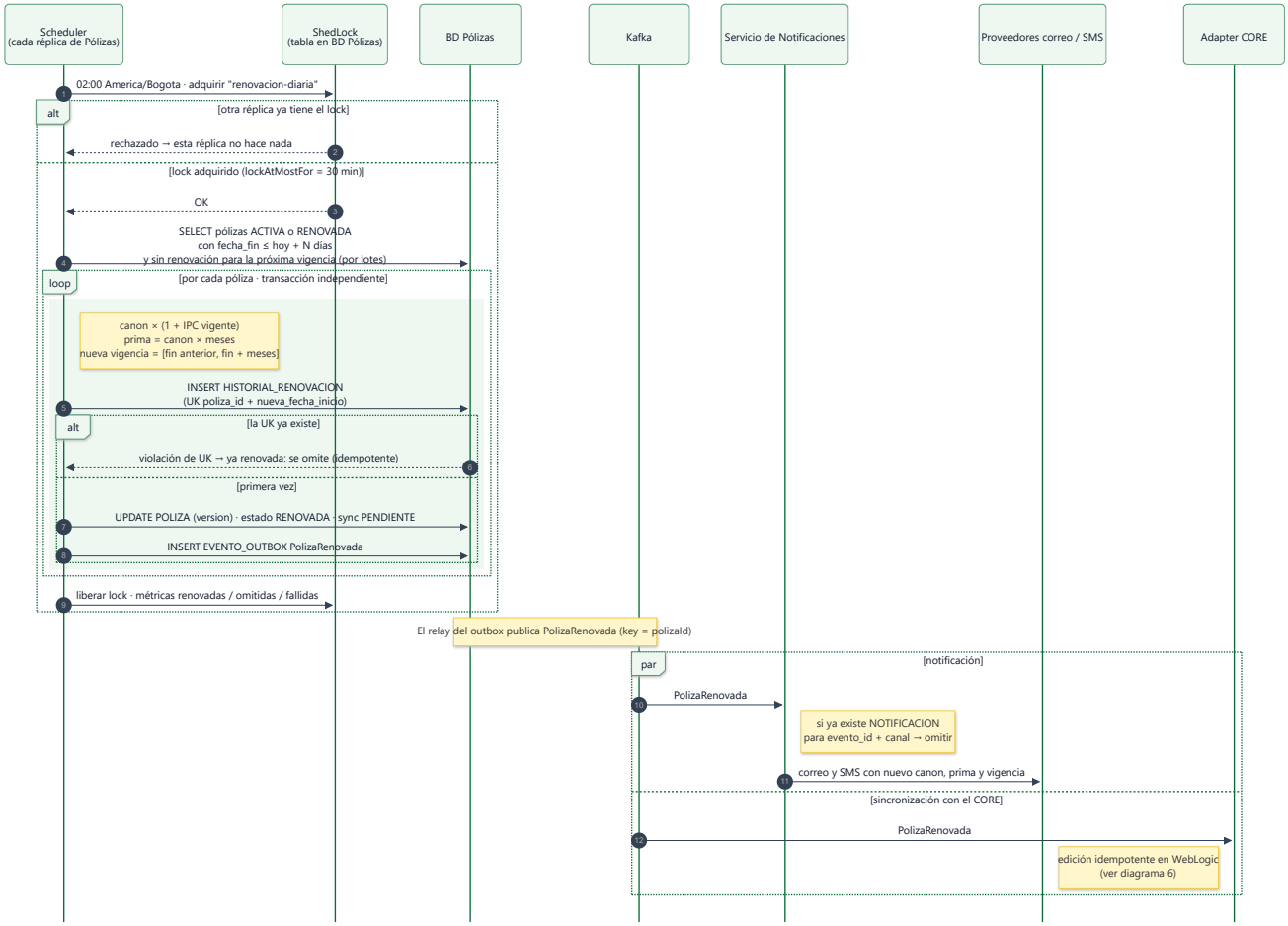


Cómo ve el usuario la sincronización con el CORE (propuesta)

> Ni la API ni el frontend del Módulo 2 tienen todavía este estado de sincronización: depende del outbox y del `estado_sync_core` propuestos (ver §8).

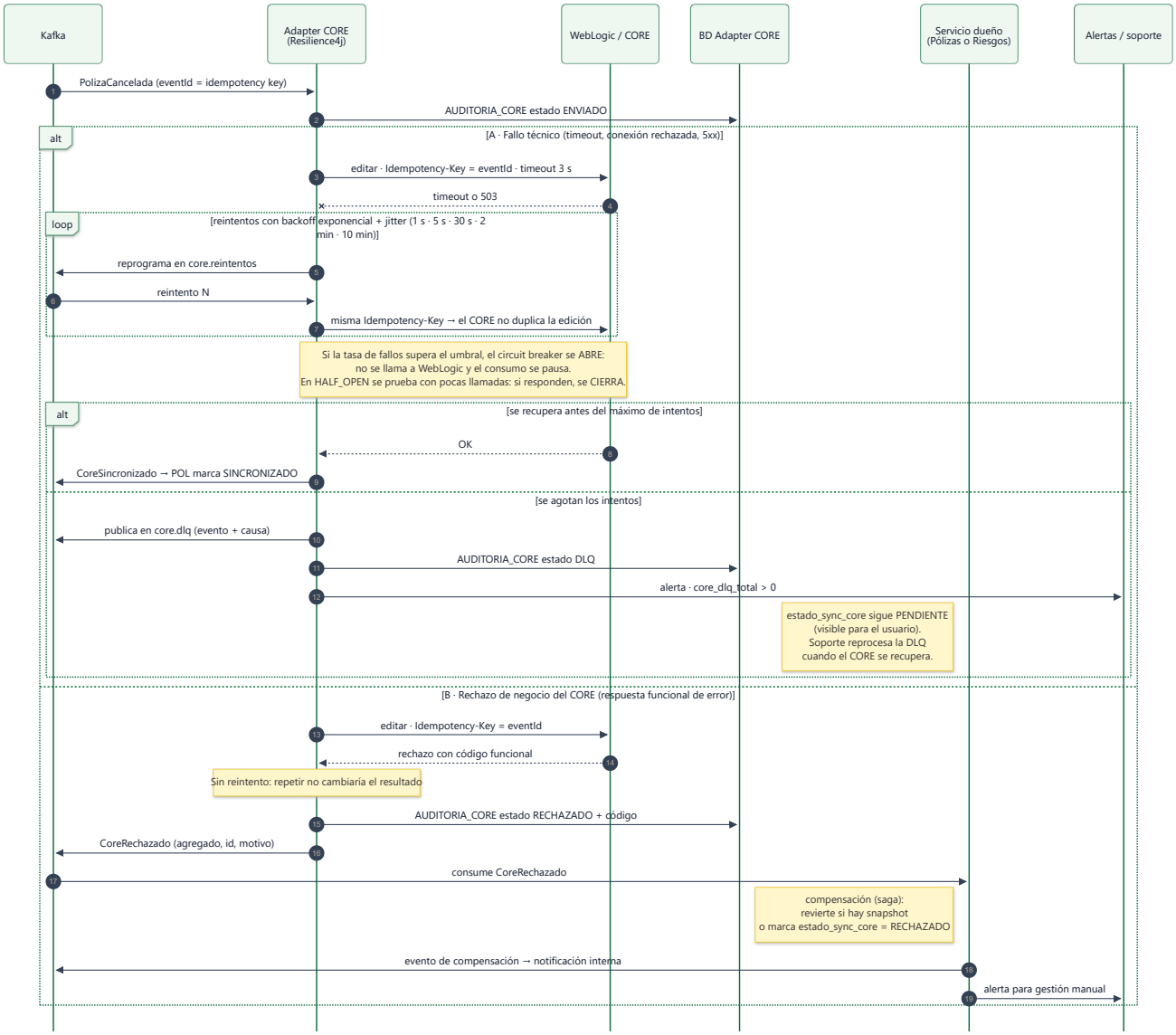
Canal	Comportamiento
API	Responde **201/200** con `estadoSincronizacion: "PENDIENTE"`. Se descarta 202: la operación **ya quedó confirmada y es durable** en la plataforma, que es la fuente de verdad; solo falta propagarla. `GET` devuelve `estadoSincronizacion` y, si fue rechazada, `motivoRechazo`.
Frontend (propuesta)	Etiqueta junto al estado: "Pendiente de sincronizar con CORE" (gris), "Sincronizado" (verde) o "Rechazado por CORE" (rojo, con el motivo y el código de seguimiento). La vista se actualiza al recargar; más adelante, con SSE si se requiere tiempo real.

5.2 Renovación automática



- **Una sola ejecución:** el job corre en todas las réplicas, pero ShedLock garantiza que solo una lo ejecute. `lockAtMostFor` libera el lock si esa réplica muere.
- **Idempotencia:** si el job se reintenta o se solapa, la clave única de `HISTORIAL_RENOVACION` impide renovar dos veces la misma vigencia.
- **Aislamiento de fallos:** cada póliza se procesa en su propia transacción, así que un fallo no detiene el lote. Las métricas `renovaciones_{ok,omitidas,fallidas}` alimentan alertas.

5.3 Fallo del CORE: técnico frente a rechazo de negocio



	A · Fallo técnico	B · Rechazo de negocio
Ejemplos	Timeout, conexión rechazada, HTTP 5xx, WebLogic caído	Código funcional de error del CORE (p. ej. la entidad no existe en el CORE o el estado no lo permite)
¿Reintentar?	**Sí**, con backoff exponencial y jitter, y siempre la misma idempotency key	**No**: repetir no cambia el resultado
Protección	Circuit breaker (abierto → pausa el consumo), bulkhead y timeout	—
Al agotar	`core.dlq` + alerta; `estado_sync_core` sigue PENDIENTE; reproceso manual	Compensación (saga), notificación interna y alerta
Estado final	SINCRONIZADO al recuperarse	RECHAZADO (o revertido)

Compensación por operación ante un rechazo de negocio

Operación	Compensación
Crear póliza	Se marca RECHAZADO; la póliza queda bloqueada para nuevas operaciones hasta gestión manual (no se borra, por auditoría)
Renovar	Se revierte a los valores y la vigencia anteriores desde `HISTORIAL_RENOVACION` (snapshot)
Cancelar póliza o riesgo	Se marca RECHAZADO; no se revierte la cancelación automáticamente (es decisión de negocio)
Agregar riesgo	El riesgo local pasa a CANCELADO con `estado_sync_core = RECHAZADO`

6. Atributos de calidad

6.1 Escalabilidad

Técnica	Aplicación
Servicios *stateless*	Sin sesión en memoria (como en el Módulo 2); cualquier réplica atiende cualquier petición
Escalado horizontal	HPA de Kubernetes por CPU y peticiones por segundo; los consumidores escalan hasta el número de particiones
Particionamiento	Kafka por `polizaId` (orden por póliza y paralelismo). Oracle con particionado por intervalo de fecha en outbox, auditoría y notificaciones (purga barata)
Réplicas de lectura	Active Data Guard para consultas y reportes
Caché	Parámetros (IPC) y catálogos en caché local (Caffeine) con TTL; las consultas de detalle se cachean solo si las métricas lo justifican
Paginación	Listados con `Pageable` (en el Módulo 2 el listado aún no está paginado)

6.2 Logs y observabilidad

Señal	Herramienta	Detalle
Logs	JSON (ECS) con `correlationId` → Loki o Elastic	**Ya implementado** en el Módulo 2 (perfil `json`)
Métricas	Micrometer + Prometheus	RED por endpoint; de negocio: antigüedad de pendientes de sincronización, rezago del outbox, tamaño de la DLQ, renovaciones
Trazas	OpenTelemetry → Tempo o Jaeger	`traceparent` también en los headers de Kafka: una traza cruza Gateway → servicio → Kafka → Adapter → WebLogic
Dashboards y alertas	Grafana	Alertas por *burn rate* de SLO, no por umbrales sueltos

SLOs propuestos (a validar con negocio):

SLO	Objetivo
Disponibilidad de la API	99,9 % de respuestas no-5xx en 30 días
Latencia de consultas	p95 < 300 ms
Sincronización con el CORE	99 % de los cambios SINCRONIZADO en menos de 5 min
Renovación automática	Job diario completado antes de las 06:00

6.3 Tolerancia a fallos

Mecanismo	Dónde
Timeout, retry con backoff y jitter, circuit breaker y bulkhead (Resilience4j)	Adapter CORE y llamadas a proveedores
Transactional Outbox	Pólizas y Riesgos
Consumidores idempotentes	Riesgos (proyección y cascada), Notificaciones, Adapter
DLQ + reproceso	`core.dlq` y una DLQ por consumidor
`Idempotency-Key` en POST públicos	Gateway y servicios: evita duplicados cuando el cliente reintenta
Health checks (liveness y readiness)	Actuator en cada servicio
Despliegue sin caída	Rolling o canary con rollback automático por SLO; blue-green para cambios de esquema
Multi-zona	Réplicas en 3 zonas, Kafka con factor de replicación 3 y `min.insync.replicas=2`

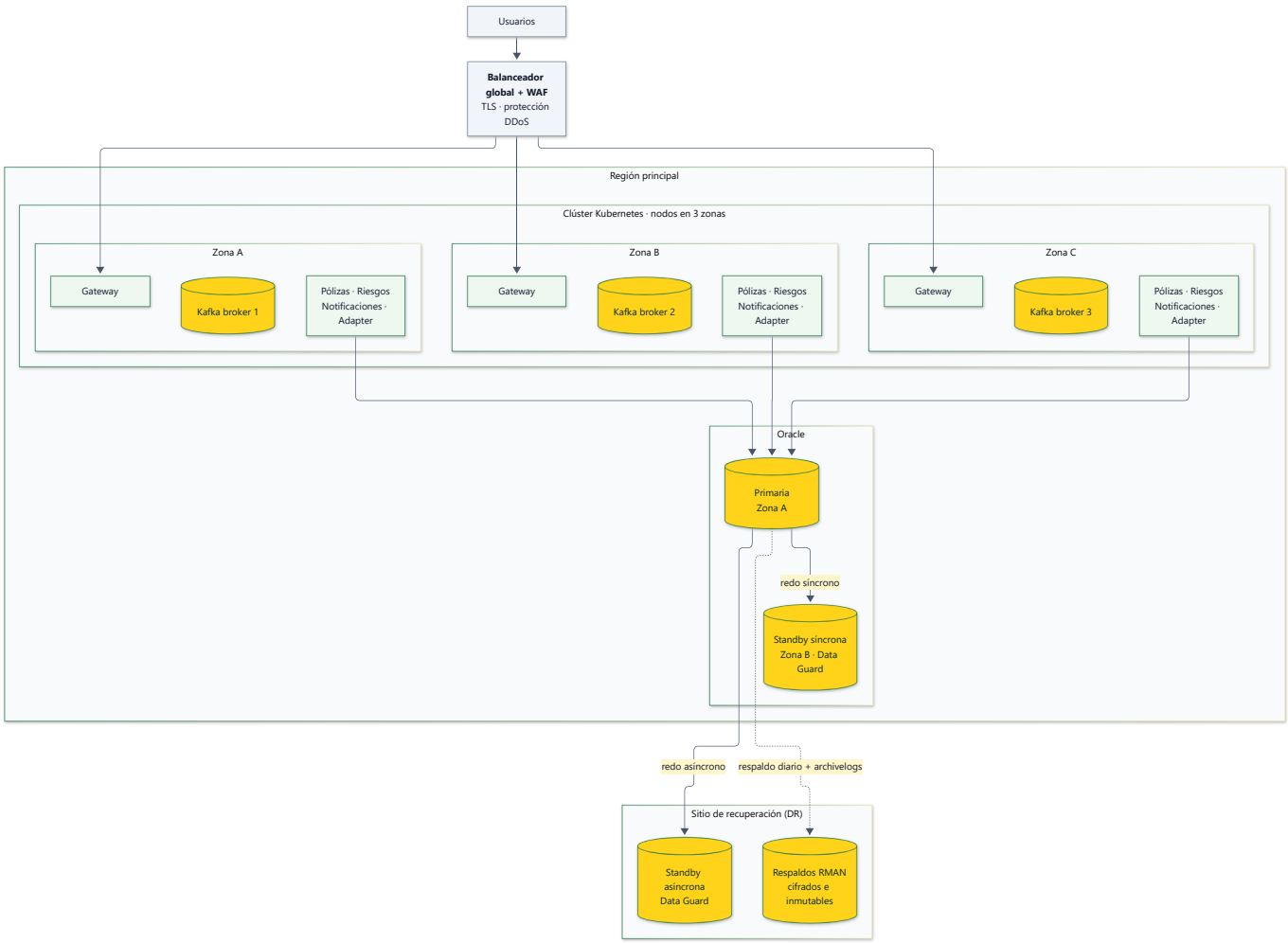
6.4 Versionamiento de APIs

Aspecto	Decisión
Estrategia	**Versión mayor en la URI** (`/api/v1/...`), gestionada en el Gateway. Es explícita, cacheable y fácil de enrutar. Se descarta versionar por header por ser menos visible para los consumidores.
Compatibilidad	Dentro de una versión mayor solo hay cambios aditivos (campos opcionales nuevos, endpoints nuevos). Romper el contrato exige `/v2` , desplegado en paralelo.
Deprecación	Headers `Deprecation` (RFC 9745) y `Sunset` (RFC 8594) más un enlace a la guía de migración. Ventana mínima propuesta: 6 meses.
Contratos	OpenAPI versionado en el repositorio (ya generado con springdoc) y pruebas de contrato con los consumidores
Eventos	Los esquemas de eventos también se versionan (Schema Registry con compatibilidad *backward*)

6.5 Seguridad

Aspecto	Propuesta
Identidad	OAuth2/OIDC: el frontend obtiene tokens del IdP (Authorization Code + PKCE). El Gateway los valida y los servicios verifican *scopes* (p. ej. `polizas:escribir`).
Entre servicios	mTLS (service mesh, p. ej. Istio o Linkerd) y principio de mínimo privilegio
Secretos	Vault o *External Secrets* (equivalentes: AWS Secrets Manager, Azure Key Vault); nunca en imágenes ni repositorios
Datos personales	Correo y teléfono cifrados en reposo, enmascarados en logs y notificaciones; cumplimiento de la Ley 1581 de 2012 (habeas data)
OWASP API Top 10	Ver Anexo B

7. Disponibilidad 24/7



Elemento	Decisión
SLA objetivo propuesto	**99,9 %** mensual (≈ 43 min de indisponibilidad al mes). No es un dato de la empresa; se valida con negocio.
Sin puntos únicos de falla	Gateway y servicios con ≥ 2 réplicas por zona en 3 zonas; Kafka con 3 brokers; Oracle primaria + standby síncrona en otra zona con *fast-start failover*
Degradación controlada	Si el CORE o WebLogic caen, la plataforma **sigue operando**: los cambios quedan PENDIENTES y se sincronizan al recuperarse. Si un proveedor de correo o SMS falla, la notificación se reintenta sin afectar la operación.
Respaldo	RMAN completo semanal, incremental diario y *archivelogs* continuos; cifrados, inmutables y en otro sitio. Restauración probada trimestralmente.
RPO / RTO propuestos	Falla de zona: **RPO 0**, RTO ≤ 15 min (automático). Falla de región: RPO ≤ 5 min (Data Guard asíncrono), RTO ≤ 1 h (procedimiento ensayado).

8. Trazabilidad: diseño frente a implementación del Módulo 2

Rutas relativas a `02-api-polizas/`. Abreviaturas: `be` = `backend/src/main/java/com/segurosbolivar/polizas`, `fe` = `frontend/src/app`.

Decisión de diseño	Estado	Dónde (Módulo 2)
Arquitectura hexagonal (dominio, aplicación, infraestructura)	**Implementado**	`be/domain`, `be/application`, `be/infrastructure`
Reglas de negocio en el agregado `Poliza`	**Implementado**	`be/domain/model/Poliza.java`
Puerto CORE + adaptador (ACL)	**Implementado**	`be/domain/port/CorePort.java`, `be/infrastructure/core/CoreHttpAdapter.java`

Decisión de diseño	Estado	Dónde (Módulo 2)
Contrato `evento = ACTUALIZACION + operacion`	**Implementado**	`be/domain/port/EventoCore.java`
Timeouts hacia el CORE	**Implementado**	`be/infrastructure/core/CoreClientConfig.java`
Llamada al CORE asíncrona vía outbox + Kafka	**Simplificado**: síncrona tras el `flush`; el fallo se registra y no revierte	`be/application/PolizaService.java`
Retry, circuit breaker, DLQ (Resilience4j)	Propuesta	—
Idempotency key hacia el CORE	Propuesta	—
`estado_sync_core` en póliza y riesgo	Propuesta	—
`estadoSincronizacion` en la API y etiqueta de sincronización en el frontend	Propuesta (el frontend actual no la muestra)	—
Eventos de dominio + notificaciones tras el commit	**Simplificado**: en memoria (`@TransactionalEventListener`), correo y SMS simulados en logs	`be/infrastructure/notificacion/NotificacionService.java`
Transactional Outbox	Propuesta	—
Microservicios separados (Pólizas, Riesgos, Notificaciones, Adapter)	**Simplificado**: monolito modular, un despliegue	Todo `be/`
Renovación con IPC configurable	**Implementado** (manual)	`be/domain/model/Poliza.java#renovar`, `GET /parametros/renovacion`
Renovación automática (scheduler + ShedLock + historial)	Propuesta	—
Bloqueo optimista (`@Version`) → 409	**Implementado**	`Poliza.java`, `Riesgo.java`, `GlobalExceptionHandler.java`
Errores RFC 9457 (ProblemDetail)	**Implementado**	`be/infrastructure/web/error/GlobalExceptionHandler.java`
`correlationId` por petición, propagado al CORE y visible en el frontend	**Implementado**	`be/infrastructure/config/CorrelationIdFilter.java`, `fe/core/http/interceptors.ts`
Logs JSON estructurados (ECS)	**Implementado**	`backend/src/main/resources/application-json.yml`
Métricas (Micrometer/Prometheus) y trazas (OpenTelemetry)	Propuesta	—
Health checks (Actuator)	Propuesta	—
Esquema versionado con Flyway, compatible con Oracle y H2	**Implementado**	`backend/src/main/resources/db/migration`
Oracle + Docker Compose	**Implementado**	`backend/docker-compose.yml`
OAuth2/OIDC en el Gateway	**Simplificado**: API key en un filtro	`be/infrastructure/security/ApiKeyFilter.java`
CORS restringido y proxy del mismo origen	**Implementado**	`SeguridadConfig.java`, `frontend/nginx/default.conf.template`
Versionamiento `/v1` en el Gateway	Propuesta (las rutas actuales no llevan versión)	—
`PERSONA` como entidad	**Simplificado**: texto	`be/domain/model/Poliza.java`, `Riesgo.java`
Paginación de listados	Propuesta	—

Decisión de diseño	Estado	Dónde (Módulo 2)
Frontend que refleja reglas sin duplicarlas	**Implementado**	`fe/features/polizas`

9. Riesgos y siguientes pasos

Riesgo	Mitigación
Contrato, errores y soporte de idempotencia de WebLogic desconocidos (S2, S3)	Prueba de concepto del Adapter contra WebLogic en el primer sprint; catálogo de códigos técnicos frente a funcionales
Orden de eventos: un reintento en `core.reintentos` podría adelantar un evento posterior de la misma póliza	El Adapter procesa en orden por `polizaId` y "estaciona" los eventos posteriores mientras haya uno pendiente de la misma póliza
Ventana de consistencia eventual Pólizas/Riesgos	Proyección con `version_origen`, *read-through* y cascada idempotente (§3.1); empezar como monolito modular si el equipo es pequeño
Costo operativo de microservicios	Plataforma común (plantillas, observabilidad y CI/CD); extracción gradual desde el monolito modular
CDC sobre Oracle (Debezium/LogMiner) complejo de operar	Empezar con un relay por consulta (`SKIP LOCKED`)
Datos personales	Cifrado, enmascaramiento y revisión de cumplimiento

Siguientes pasos.

1. Prueba de concepto del Adapter CORE con el contrato real.
2. Agregar outbox, `estado_sync_core` y Resilience4j al Módulo 2 (monolito modular).
3. Actuator, Micrometer y OpenTelemetry.
4. Prefijo `/v1` y OIDC.
5. Scheduler de renovación con ShedLock.
6. Pruebas de carga para dimensionar (S1).
7. Extraer Notificaciones y luego Riesgos según métricas.

Anexo A: Catálogo de eventos

Evento	Productor	Consumidores	Clave
`PolizaCreada` (incluye riesgos iniciales)	Pólizas	Riesgos (crea riesgos y proyección), Notificaciones, Adapter CORE	`polizaId`
`PolizaRenovada`	Pólizas	Riesgos (proyección), Notificaciones, Adapter CORE	`polizaId`
`PolizaCancelada`	Pólizas	Riesgos (cascada), Adapter CORE	`polizaId`
`RiesgoAgregado` / `RiesgoCancelado`	Riesgos	Adapter CORE	`polizaId`
`CoreSincronizado` / `CoreRechazado`	Adapter CORE	Pólizas o Riesgos (estado de sincronización y compensación)	`polizaId`

Todos los eventos llevan `eventId` (UUID, idempotencia), `correlationId`, `traceparent`, `version` del esquema y `ocurridoEn`.

Anexo B: OWASP API Security Top 10 (2023)

Riesgo	Mitigación
API1 Autorización a nivel de objeto	Verificar que el usuario (p. ej. una inmobiliaria) solo accede a sus pólizas
API2 Autenticación rota	OIDC en el Gateway, tokens de vida corta, sin credenciales en el frontend
API3 Autorización a nivel de propiedad	DTOs explícitos de entrada y salida (records en el Módulo 2), sin *mass assignment*
API4 Consumo de recursos sin límite	Rate limiting en el Gateway, paginación, límites de tamaño (p. ej. máx. 100 riesgos por petición, implementado)
API5 Autorización a nivel de función	*Scopes* por operación (renovar y cancelar requieren un rol específico)
API6 Acceso sin restricción a flujos de negocio	Idempotency-Key y límites por cliente en creación
API7 SSRF	La URL de WebLogic está en configuración, nunca la envía el cliente

Riesgo	Mitigación
API8 Configuración insegura	Sin stack traces (implementado), CORS restringido (implementado), cabeceras de seguridad en nginx (implementado)
API9 Inventario deficiente	OpenAPI versionado y política de deprecación
API10 Consumo inseguro de APIs	El Adapter valida y traduce las respuestas del CORE (ACL), con timeouts y circuit breaker

Anexo C: Equivalencias por proveedor de nube

Componente	Agnóstico (propuesto)	AWS	Azure
Orquestación	Kubernetes	EKS	AKS
API Gateway	Kong o Spring Cloud Gateway	Amazon API Gateway	Azure API Management
Broker	Apache Kafka (o RabbitMQ)	Amazon MSK	Event Hubs (API Kafka)
Base de datos	Oracle + Data Guard	Amazon RDS for Oracle	Oracle Database@Azure
Identidad	IdP corporativo o Keycloak	Amazon Cognito	Microsoft Entra ID
Secretos	HashiCorp Vault	Secrets Manager	Key Vault
Observabilidad	OpenTelemetry, Prometheus, Grafana, Loki, Tempo	CloudWatch, X-Ray	Azure Monitor, Application Insights

> **RabbitMQ como alternativa a Kafka:** es más simple de operar para colas de trabajo, pero Kafka ofrece orden por clave, retención y reproceso de eventos (útil para reconstruir la proyección de Riesgos o reprocesar el CORE). Por eso se prefiere Kafka.

Módulo 2: Prueba técnica práctica (API de pólizas)

API REST en Spring Boot 3.5 (Java 17) y SPA en Angular 21 para gestionar pólizas de arrendamiento (individuales y colectivas) y sus riesgos: consulta, creación, renovación con IPC y cancelación. Cada operación que cambia el estado de una póliza o de un riesgo se notifica al sistema CORE.

Respuesta corta. El backend es la fuente de verdad de las reglas de negocio y las concentra en el modelo de dominio; el frontend solo presenta, valida formato y deshabilita acciones. Ambos se levantan con un solo `docker compose up --build`. Está cubierto con 96 pruebas de backend, 80 unitarias de frontend y 10 e2e.

Documentos detallados: backend/README.md · frontend/README.md. Las decisiones técnicas, con su alternativa descartada, están en backend/DECISIONES.md y frontend/DECISIONES.md.

Cómo ejecutarlo

Requiere Docker Desktop:

```
cd 02-api-polizas
docker compose up --build
```

Servicio	URL
Frontend	http://localhost:4200
API (Swagger UI)	http://localhost:8080/swagger-ui.html
API key de prueba	`123456` (header `x-api-key`)

Si los puertos están ocupados: `WEB_PORT=4300 API_PORT=8082 docker compose up --build`. Para detener: `docker compose down`.

También se puede ejecutar cada parte por separado (backend con H2 en memoria, `./mvnw spring-boot:run`; frontend con `npm ci && npm start`); los pasos están en cada README.

Endpoints

Método	Ruta	Descripción	Éxito
GET	`/polizas?tipo=&estado=`	Lista pólizas (filtros opcionales)	200
GET	`/polizas/{id}`	Consulta una póliza	200
POST	`/polizas`	Crea una póliza con sus riesgos	201
GET	`/polizas/{id}/riesgos`	Lista los riesgos de una póliza	200
POST	`/polizas/{id}/renovar`	Renueva aplicando el IPC	200
POST	`/polizas/{id}/cancelar`	Cancela la póliza y todos sus riesgos	200
POST	`/polizas/{id}/riesgos`	Agrega un riesgo (solo COLECTIVA)	201
POST	`/riesgos/{id}/cancelar`	Cancela un riesgo	200
GET	`/parametros/renovacion`	IPC vigente que se aplicará al renovar	200
POST	`/core-mock/evento`	Mock del CORE: registra el evento en logs	202

Reglas de negocio (viven en el dominio)

- Una póliza **INDIVIDUAL** tiene exactamente un riesgo; una **COLECTIVA** admite cero o más.
- **Prima = canon × meses de vigencia**, con `BigDecimal` y redondeo `HALF_EVEN`. Nunca `double`.
- **Renovar** aplica el IPC al canon (`canon × (1 + IPC)`), recalcula la prima y extiende la vigencia; una póliza cancelada no se renueva.
- **Cancelar** una póliza cancela también todos sus riesgos; cancelar algo ya cancelado es un conflicto.
- Agregar un riesgo a una **INDIVIDUAL** es una regla de negocio violada (422); a una póliza **cancelada** es un conflicto de estado (409).
- Toda modificación de estado de pólizas o riesgos invoca el CORE a través del puerto `CorePort`.

Arquitectura (hexagonal ligera)

Capa	Contenido
`domain/`	Entidades con las reglas, excepciones de negocio, eventos de dominio y puertos. Sin dependencias de web ni de Spring (salvo anotaciones JPA).
`application/`	Casos de uso (`PolizaService` , `RiesgoService`): cargan, invocan el dominio, guardan y notifican al CORE.
`infrastructure/`	Web (controllers, DTOs, mappers, errores), persistencia JPA, adaptador del CORE, notificaciones, seguridad y configuración.

Errores y observabilidad

Todas las respuestas de error siguen RFC 9457 – Problem Details e incluyen un `correlationId` para buscar la traza en los logs.

Código	Cuándo
400	Campos inválidos (con detalle por campo), JSON mal formado o parámetros con tipo inválido
401	API key ausente o incorrecta
404	La póliza, el riesgo o la ruta no existe
409	Conflicto con el estado actual (cancelar algo ya cancelado, renovar una cancelada, modificación concurrente)
422	Regla de negocio violada por la petición (riesgo en una INDIVIDUAL, INDIVIDUAL sin exactamente un riesgo)

Cada petición recibe un `correlationId` que se devuelve en la respuesta, aparece en los logs y se propaga al CORE. Los logs registran solo identificadores: nunca nombres de tomadores, direcciones ni la API key.

Frontend

SPA Angular 21 (standalone, *zoneless* y *signals*) que consume la API a través del proxy `/api` (el navegador nunca conoce la URL del backend).

- **Listado** con filtros por tipo y estado (los filtros van en la URL, se pueden compartir).
- **Detalle** con datos, vigencia, canon, prima y tabla de riesgos; acciones según el estado y el tipo.
- **Creación** de INDIVIDUAL (un riesgo) y COLECTIVA (`FormArray` , 0..n) con prima estimada y errores 400 del backend en cada campo.
- **Renovación** en un diálogo que muestra el IPC y compara los valores anteriores frente a los nuevos.
- **Cancelación** de póliza y de riesgo con confirmación (`<dialog>` nativo accesible).

Estados de carga, vacío y error en cada vista; accesibilidad obligatoria (etiquetas, `aria-*` , contraste AA).

Pruebas

Ámbito	Herramienta	Cantidad
Backend (dominio, servicios, repositorio, integración con MockMvc)	JUnit 5, Mockito, JaCoCo	96 pruebas · 99,8 % de líneas y 91 % de ramas
Frontend unitarias	Vitest	80 pruebas
Frontend e2e	Playwright (Chromium)	10 flujos sobre la aplicación real

El build del backend **falla** si la cobertura de líneas baja del 80 %. La colección de Postman (backend/postman/) incluye 20 peticiones con aserciones. El detalle de cada suite está en el README correspondiente.

Documentos relacionados

- backend/README.md · backend/DECISIONES.md
- frontend/README.md · frontend/DECISIONES.md
- Diseño de la solución y trazabilidad: ../01-diseno-sistema/ARQUITECTURA.md

Módulo 3: Conocimientos en base de datos

> **Enunciado.** `orders` tiene 10 millones de registros y `customers` 500.000. Esta consulta es muy lenta y se ejecuta con frecuencia; se piden al menos tres estrategias de optimización.
>
> ```
> SELECT o.order_id, o.order_date, c.customer_name, o.total_amount
> FROM orders o JOIN customers c ON o.customer_id = c.customer_id
> WHERE c.country = 'México';
> ```

Respuesta corta. Primero diagnostico con el plan real (filas estimadas frente a reales). Después aplico, en este orden: estadísticas con histograma en `country`, índices compuestos cubrientes, rediseño de la consulta (paginación por *keyset* y rango de fechas) y, si es un reporte, particionamiento o vista materializada. En la demo (2M orders / 100k customers, Oracle 23.26 Free), la paginación por *keyset* bajó la consulta de **226 ms y 42.073 lecturas a menos de 1 ms y 502 lecturas**, y el índice redujo un **86 %** el tiempo cuando el país es poco frecuente.

1. Diagnóstico primero

Optimizar sin mirar el plan es adivinar. Uso el plan **real**, no solo el estimado:

```
-- Plan estimado (no ejecuta la consulta)
EXPLAIN PLAN FOR SELECT ...;
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);

-- Plan REAL de la última ejecución, con filas estimadas frente a reales
SELECT /*+ gather_plan_statistics */ o.order_id, ... ;
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY_CURSOR(NULL, NULL, 'ALLSTATS LAST'));
```

Qué mirar	Señal de problema
`TABLE ACCESS FULL` sobre `orders`	Se leen las 10M filas para devolver una fracción
E-Rows frente a A-Rows	Si difieren mucho, el optimizador decide con datos equivocados (estadísticas desactualizadas o falta de histograma)
Tipo de join	`NESTED LOOPS` es bueno para pocas filas; `HASH JOIN` lo es para muchas. Un join inadecuado suele venir de una mala estimación
`Buffers` y `Reads`	Lecturas lógicas y físicas: el costo real en bloques
Sección *Predicate Information*	Funciones sobre columnas (`UPPER(country)`) o conversiones implícitas que impiden usar índices

MySQL: `EXPLAIN ANALYZE` (8.0.18+) muestra filas estimadas y reales; `EXPLAIN FORMAT=TREE` muestra el tipo de join.

2. Estrategias

2.1 Estadísticas actualizadas e histograma en `country`

Qué resuelve. La distribución por país está sesgada. Sin histograma, Oracle supone que todos los países pesan igual (1 / número de valores distintos) y estima mal la cardinalidad, lo que lleva a elegir mal el join y el método de acceso.

```
BEGIN
  DBMS_STATS.GATHER_TABLE_STATS(USER, 'CUSTOMERS',
    method_opt => 'FOR ALL COLUMNS SIZE 1 FOR COLUMNS country SIZE 254', cascade => TRUE);
  DBMS_STATS.GATHER_TABLE_STATS(USER, 'ORDERS', cascade => TRUE);
END;
/
```

- **Impacto medido:** las estimaciones pasaron de 10.000 clientes para **cualquier** país a 19.027 (México) y 2.039 (Costa Rica), iguales a las reales (§4.3). Sin índices el plan no cambió (no hay alternativa a los *full scans*), pero es el **requisito** para que las demás estrategias se elijan bien.
- **Costo:** bajo; debe programarse (el job automático de estadísticas de Oracle lo hace en ventanas nocturnas).
- *MySQL 8:* `ANALYZE TABLE customers UPDATE HISTOGRAM ON country WITH 64 BUCKETS;`

2.2 Índices compuestos y cubrientes

Qué resuelve. El filtro por país y el join por `customer_id` sin índices obligan a recorrer ambas tablas completas. Un índice **cubriente** contiene todas las columnas que la consulta necesita, así que no hace falta visitar la tabla.

```
CREATE INDEX ix_customers_pais ON customers (country, customer_id, customer_name);
CREATE INDEX ix_orders_cliente ON orders (customer_id, order_date, total_amount, order_id);
```

`order_id` se agrega al final para que el índice cubra todas las columnas del `SELECT`. Además, la FK `orders.customer_id` queda indexada (Oracle no lo hace automáticamente).

- **Impacto medido:** depende de la **selectividad** (§4.3). Con Costa Rica (2 %): de 134 ms a **19 ms** (−86 %) con *nested loops*. Con México (19 %): las lecturas bajan un 76 %, pero el tiempo no mejora (*hash join* sobre el índice).
- **Costo:** espacio (75 MB a esta escala) y escrituras más lentas en `orders` (cada inserción mantiene un índice más).
- *MySQL (InnoDB):* igual. Como el índice secundario incluye la PK implícitamente, `(customer_id, order_date, total_amount)` ya cubre `order_id`; `EXPLAIN` muestra `Using index`.

2.3 Particionamiento de `orders`

Qué resuelve. Con 10M de filas y crecimiento continuo, la mayoría de consultas operativas miran un periodo reciente. Particionar por rango de `order_date` permite el **partition pruning**: leer solo las particiones del periodo.

```
CREATE TABLE orders_part
  PARTITION BY RANGE (order_date) INTERVAL (NUMTOYMINTERVAL(1, 'MONTH'))
  (PARTITION p_inicial VALUES LESS THAN (DATE '2022-01-01'))
  AS SELECT * FROM orders;
CREATE INDEX ix_orders_part_cliente ON orders_part (customer_id, order_date, total_amount, order_id) LOCAL;
-- Variante compuesta: ... SUBPARTITION BY HASH (customer_id) SUBPARTITIONS 8 (paraleliza los joins por cliente)
```

- **Impacto medido** (consulta acotada al último trimestre): de 81 ms y 9.761 lecturas a **22 ms y 758 lecturas** (−73 % / −92 %). Se leyeron 3 de 49 particiones.
- **Limitación:** si la consulta **no** filtra por fecha, el particionamiento por fecha no ayuda. Es una estrategia para combinar con 2.5.
- **Costo:** licencia (en Oracle Enterprise es una opción de pago), una migración de la tabla y disciplina en las consultas. A cambio, se pueden archivar o purgar particiones antiguas con `DROP PARTITION` en segundos.
- *MySQL:* `PARTITION BY RANGE (YEAR(order_date))…`, con la restricción de que la clave de partición debe estar en toda clave única.

2.4 Vista materializada

Qué resuelve. Si la consulta es de **reporte** (no transaccional), se puede pre-calcular el join y leer el resultado.

```
CREATE MATERIALIZED VIEW LOG ON orders WITH ROWID;
CREATE MATERIALIZED VIEW LOG ON customers WITH ROWID;
CREATE MATERIALIZED VIEW mv_ordenes_cliente
  BUILD IMMEDIATE REFRESH FAST ON DEMAND        -- o ON COMMIT; o COMPLETE programado con DBMS_SCHEDULER
  AS SELECT o.ROWID o_rid, c.ROWID c_rid, o.order_id, o.order_date, c.customer_name, o.total_amount, c.country
     FROM orders o, customers c WHERE o.customer_id = c.customer_id;
CREATE INDEX ix_mv_pais ON mv_ordenes_cliente (country);
-- Refresco incremental:  EXEC DBMS_MVIEW.REFRESH('MV_ORDENES_CLIENTE', 'F');
```

- **Impacto medido:** México de 226 ms a **126 ms** (−44 %). El refresco FAST tras insertar una orden tomó **0,28 s** y la vista quedó `FRESH`.
- **Costo:** duplica los datos (otra copia de 2M filas), los MV logs agregan trabajo a **cada** DML en `orders` y `customers`, y los datos tienen el retraso del último refresco.
- *MySQL:* no tiene vistas materializadas nativas; se emulan con una tabla resumen mantenida por eventos (`CREATE EVENT`) o por la aplicación.

2.5 Diseño de la consulta y de los datos

Qué resuelve. La consulta devuelve **todas** las órdenes de un país: 379.645 filas a esta escala, unas 1,9 millones con 10M. Ninguna pantalla consume eso. Si se ejecuta "con frecuencia", casi seguro alimenta una vista paginada o un periodo concreto.

```
-- a) Paginación keyset (índice por fecha): cada página cuesta lo mismo, sin importar su número
CREATE INDEX ix_orders_fecha ON orders (order_date, order_id, customer_id, total_amount);

SELECT o.order_id, o.order_date, c.customer_name, o.total_amount
FROM orders o JOIN customers c ON o.customer_id = c.customer_id
WHERE c.country_code = 'MX'
  AND (o.order_date < :ult_fecha OR (o.order_date = :ult_fecha AND o.order_id < :ult_id))  -- "después de" la
última fila vista
ORDER BY o.order_date DESC, o.order_id DESC
FETCH FIRST 50 ROWS ONLY;

-- b) Rango de fechas obligatorio para listados y reportes:  AND o.order_date >= :desde
-- c) Solo las columnas necesarias (nunca SELECT *): habilita los índices cubrientes
```

- **Impacto medido:** primera página y página 201 por *keyset* en **menos de 1 ms** (518 y 502 lecturas). La misma página 201 con `OFFSET 10000` tardó 220 ms y 9.737 lecturas, porque `OFFSET` tiene que producir y descartar las 10.000 filas anteriores.

Normalizar `country`. El texto libre produce variantes: en la demo, **1.025 clientes** guardados como `Mexico` o `MEXICO` (20.371 órdenes) quedan **fuera** del resultado de `= 'México'`. No es un problema de rendimiento, sino de **datos incorrectos**. Además, `WHERE UPPER(country) = …` impediría usar el índice.

```
ALTER TABLE customers ADD country_code CHAR(2);    -- ISO 3166-1 alfa-2, idealmente con FK a un catálogo de
países
-- migración: UPDATE customers SET country_code = <mapeo normalizado>;  luego NOT NULL + índice (country_code,
customer_id, customer_name)
```

En la demo, `country_code = 'MX'` encontró **20.052** clientes, frente a 19.027 con el texto exacto.

3. Resumen y prioridad recomendada

#	Estrategia	Cuándo usarla	Costo	Prioridad
1	**Diagnóstico + estadísticas con histograma**	Siempre, antes que nada	Bajo	**1**
2	**Diseño de la consulta** (paginación *keyset*, rango de fechas, columnas necesarias) y **normalizar `country`**	Consultas de pantallas y APIs; datos de calidad dudosa	Bajo a medio (cambio en la aplicación y migración de datos)	**2**: el mayor impacto medido
3	**Índices compuestos y cubrientes**	Filtros selectivos (pocos clientes por país) y joins por FK	Espacio y escrituras más lentas	**3**
4	**Particionamiento** por fecha (y hash por cliente)	Tablas grandes que crecen, consultas por periodo, archivado	Licencia y migración	4: cuando el volumen lo justifique
5	**Vista materializada**	Reportes que toleran datos con algo de retraso	Almacenamiento y DML más costoso	5: solo para reportes

4. Demostración con datos reales

4.1 Entorno, escala y método

Aspecto	Detalle
Motor	Oracle Database 23.26 Free (imagen `gvenzl/oracle-free:23-slim`, del `docker-compose.yml` del backend). Limitado por licencia a 2 CPU y 2 GB de RAM.
Máquina	Laptop Windows 11 con Docker Desktop
Escala	**2.000.000 orders / 100.000 customers**: un quinto del enunciado, con la misma proporción 20:1. Se genera en ~25 s y permite repetir la demo en minutos. Con `ORDERS=10000000 CUSTOMERS=500000` se corre a la escala completa.
Datos	Semilla fija (reproducible). Países: Colombia 45 %, **México 20 %** (19,0 % como "México", 1 % como "Mexico" o "MEXICO"), Perú 10 %, Chile 8 %, Ecuador 7 %, Argentina 5 %, Panamá 3 %, **Costa Rica 2 %**. Orders: cliente uniforme, fechas 2022–2025. Tamaño: `orders` 312 MB, `customers` 20 MB.
Medición	Por consulta: 1 ejecución de calentamiento + 5 medidas, **trayendo todas las filas**. Se reporta la mediana del tiempo y las lecturas lógicas (`session logical reads`). Plan real con `DISPLAY_CURSOR(…'ALLSTATS LAST')` .
Advertencia	Los datos caben en la caché (lecturas físicas ≈ 0). Los **tiempos absolutos** dependen de la máquina; las **lecturas lógicas** y la **forma del plan** son la señal comparable. Cada estrategia se aplica sobre la anterior.

4.2 Resultados (corrida del 2026-10-15)

Paso	Escenario	Plan (operación principal)	Mediana ms	Lecturas lógicas	Filas
E0 Línea base (solo PK, sin histograma)	México (19 %)	Hash join + full scan de ambas tablas	226	42.073	379.645
	Costa Rica (2 %)	Hash join + full scan de ambas tablas	134	41.740	40.959
E1 + histograma en `country`	México	Igual (estimación corregida)	231	42.073	379.645
	Costa Rica	Igual (estimación corregida)	133	41.740	40.959
E2 + índices cubrientes	México	**Hash join** + *index fast full scan*	239	**10.116**	379.645
	Costa Rica	**Nested loops** + *index range scans*	**19**	**4.413**	40.959
E3 Último trimestre, sin particionar	México, `order_date ≥ 2025-10-01`	Hash join + *index fast full scan*	81	9.761	24.143
E3 Último trimestre, particionada	México, `order_date ≥ 2025-10-01`	*Partition range iterator* (3 de 49)	**22**	**758**	24.143
E4 Vista materializada	México	Full scan de la MV (sin join)	126	19.737	379.645
	Costa Rica	Full scan de la MV	49	19.400	40.959
E5 Paginación: página 1	México, 50 filas	Índice por fecha descendente + nested loops	**< 1**	518	50
E5 Página 201 con `OFFSET 10000`	México, 50 filas	Hash join + ordenamiento de 379K filas	220	9.737	50
E5 Página 201 con *keyset*	México, 50 filas	Índice por fecha + nested loops	**< 1**	502	50

Tiempos de creación: `ix_orders_cliente` 2,2 s; tabla particionada 2,2 s + índice local 2,4 s; vista materializada 2,8 s; refresco FAST 0,28 s; `ix_orders_fecha` 5,3 s. Salida completa con todos los planes: `demo/resultados/salida-completa.txt` .

4.3 Efecto de la selectividad: nested loops frente a hash join

Sin histograma, la estimación es ciega al país. Para cualquier país, Oracle estimó 10.000 clientes (100.000 / 10 valores distintos):

	México: estimado → real	Costa Rica: estimado → real
Clientes del país (E0, sin histograma)	10.000 → 19.027 (subestima ×1,9)	10.000 → 2.039 (sobreestima ×4,9)
Filas del join (E0)	198K → 379K	198K → 41K
Con histograma (E1)	19.027 → 19.027; join 377K → 379K	2.039 → 2.039; join 40.447 → 40.959

Con índices, el optimizador cambió de estrategia según la selectividad, y en ambos casos eligió bien:

- **Costa Rica (2 %, 2.039 clientes): `NESTED LOOPS`.** Por cada cliente hace un *range scan* en `ix_orders_cliente` (2.039 búsquedas, unas 2 lecturas cada una). Total: **4.413 lecturas**, frente a las ~10.000 que costaría recorrer todo el índice. Por eso bajó de 134 a 19 ms.
- **México (19 %, 19.027 clientes): `HASH JOIN`.** Hacer 19.027 búsquedas costaría unas 40.000 lecturas. Le sale más barato recorrer **una vez** el índice cubriente completo (*fast full scan*, ~10.000 bloques), que ocupa la cuarta parte de la tabla: de 42.073 a 10.116 lecturas.
- **Punto de cruce aproximado:** estimado a partir de estas mediciones (no medido directamente), está alrededor de 10.000 / 2 ≈ **5.000 clientes, ~5 % del total**. Por debajo conviene el índice con *nested loops*; por encima, el *hash join*. Un índice no es "siempre más rápido": su beneficio depende de cuántas filas filtra.
- **Por qué México no bajó de tiempo pese al −76 % de lecturas:** con todo en caché, el costo dominante es construir el hash, cruzar y **transferir 379.645 filas**, no leer bloques. En producción, con caché fría y concurrencia, leer 4 veces menos bloques sí reduce E/S y presión sobre la caché. Pero la conclusión de fondo se mantiene: para una consulta que devuelve el 19 % de 10M de filas, la solución real es **no devolverlas todas** (§2.5), y los números de E5 lo confirman.
- **La vista materializada** (E4) también es *full scan* para Costa Rica. La vista no tenía histograma (estimó 200K filas), pero aun con una estimación correcta, leer ~41K filas dispersas por índice tocaría más bloques que el recorrido completo (19.400). Se mejoraría ordenando o particionando la vista por país (*attribute clustering*).

4.4 Cómo reproducir

```
cd 03-base-datos/demo
./ejecutar-demo.sh                # Linux / macOS / Git Bash
.\ejecutar-demo.ps1               # Windows PowerShell
ORDERS=10000000 CUSTOMERS=500000 ./ejecutar-demo.sh   # escala del enunciado
MANTENER=1 ./ejecutar-demo.sh     # conservar datos y contenedor para explorar
```

El script levanta el servicio `oracle` del compose del backend, crea el usuario `demo_perf`, genera los datos, ejecuta los experimentos y guarda la salida en `demo/resultados/`. Al terminar elimina el usuario y detiene el contenedor. Scripts SQL en `demo/sql/`.

La versión `.sh` se ejecutó completa a escala 2M/100k (los resultados de esta sección). La `.ps1` se probó a escala reducida (100k/5k) y terminó sin errores.

Módulo 4: Conocimientos en versionamiento

> **Enunciado.** Estoy trabajando en `feature/new-login`. Un compañero fusionó en `main` un cambio crítico que corrige un bug de seguridad, y necesito incorporar **solo ese cambio** a mi rama, sin el resto de `main`. ¿Qué comando o estrategia usaría y por qué?

Respuesta corta. Uso `git cherry-pick -x <hash-del-fix>`. Aplica en mi rama únicamente los cambios de ese commit, como un commit nuevo que referencia al original. `git merge main` y `git rebase main` traerían **todo** lo nuevo de `main`, que es justo lo que se quiere evitar.

1. Por qué cherry-pick

Opción	¿Trae solo el fix?	Comentario
`git cherry-pick -x <hash>`	**Sí**	Copia el *diff* de ese commit; `-x` deja trazabilidad al commit original
`git merge origin/main`	No	Integra todos los commits nuevos de `main`
`git rebase origin/main`	No	Mueve la rama sobre todo `main` y además reescribe la historia (requiere *force push*)

2. Paso a paso

```
# 1. Actualizar las referencias remotas (sin tocar mi rama)
git fetch origin

# 2. Identificar el commit del fix en main (por mensaje, autor, PR o archivo)
git log origin/main --oneline --grep="security"
git log origin/main --oneline -- src/auth/          # o por la ruta que corrige
git show <hash>                                     # revisar qué cambia exactamente

# 3. Aplicarlo en mi rama, con trazabilidad
git switch feature/new-login
git cherry-pick -x <hash>
#   el mensaje queda: "... (cherry picked from commit <hash>)"

# 4. Verificar: compilar y correr las pruebas (incluidas las de seguridad)
./mvnw verify            # o npm test, según el proyecto

# 5. Publicar: es un commit nuevo encima de mi rama, no requiere --force
git push
```

3. Casos especiales

Caso	Qué hacer
El fix llegó como *merge commit* (PR con *merge*)	`git cherry-pick -x -m 1 <hash-del-merge>`. `-m 1` indica que el diff se calcula contra el primer padre (la línea de `main`), es decir, se aplican todos los cambios que trajo ese PR.
Fueron varios commits	Rango consecutivo: `git cherry-pick -x A^..B` (incluye A y B; `A..B` excluiría A). Si no son consecutivos: `git cherry-pick -x <h1> <h2> <h3>`, en orden cronológico.
Hay conflictos	Git se detiene. Resuelvo los archivos, `git add <archivos>` y `git cherry-pick --continue`. Para deshacer todo, `git cherry-pick --abort`; para saltar un commit de un rango, `git cherry-pick --skip`.
El PR se integró con *squash merge*	Es el caso más simple: el fix es **un solo commit normal** en `main`. Se aplica con `git cherry-pick -x <hash>` sin `-m`.

4. Riesgos y buenas prácticas

- **Dependencias ocultas.** El fix puede usar código introducido por *otros* commits de `main` (una función utilitaria o una nueva versión de una librería). Por eso reviso `git show <hash>`, compilo y corro las pruebas antes de publicar. Si depende de otros cambios, los traigo también, de forma explícita y mínima.
- **El commit queda "duplicado" (con otro hash), y Git lo maneja.** Cuando la feature se integre en `main`, el merge de tres vías ve el mismo cambio en ambos lados y no genera conflicto. Si en lugar de merge se hace `git rebase main`, Git detecta que el parche ya existe (mismo *patch-id*) y lo omite. `git cherry -v main feature/new-login` marca con `-` los commits equivalentes. La demo (§6) lo comprueba. Solo puede haber conflicto si en `main` volvieron a modificar esas mismas líneas después del fix.
- **Trazabilidad con `-x`.** Cualquiera que vea el commit en la feature sabe de dónde viene; sirve en auditorías de seguridad.
- **Comunicación.** Lo menciono en la descripción del PR de la feature ("incluye cherry-pick de `<hash>`, fix SEC-123") para que el revisor no lo confunda con trabajo propio de la rama.
- **Pruebas de seguridad.** Además de las pruebas unitarias, corro las pruebas o el escaneo que cubren la vulnerabilidad (SAST o la prueba de regresión que haya agregado el fix).
- **No reemplaza la integración normal.** Más adelante la feature se actualizará con `main` o se integrará por PR; el cherry-pick resuelve la urgencia, no sustituye ese flujo.

5. Alternativas y por qué no

Alternativa	Por qué no en este caso
`git merge origin/main`	Trae todo `main`: cambios ajenos y quizá inestables para la feature. Es válida cuando sí se quiere actualizar la rama completa, pero no es lo que pide el escenario.
`git rebase origin/main`	Igual que el merge en alcance, y además reescribe la historia de una rama ya publicada (*force push*, que afecta a quien comparta la rama).
`git checkout origin/main -- archivo`	Copia el **archivo completo** como está en `main`, incluidas otras modificaciones a ese archivo que no son del fix. Pierde autor, mensaje y trazabilidad.
`git diff A B | git apply` o `format-patch` / `am`	Funciona, pero es una versión manual de lo que `cherry-pick` ya hace con metadatos, detección de conflictos y referencia al original. Útil solo entre repositorios distintos.

6. Demo reproducible

Scripts: `demo-cherry-pick.sh` (Linux, macOS, Git Bash) y `demo-cherry-pick.ps1` (Windows PowerShell). Cada uno:

1. Crea un repositorio temporal.
2. Simula el escenario: `main` recibe un cambio no relacionado, el fix de seguridad y otro cambio no relacionado, mientras la feature avanza.
3. Aplica el cherry-pick y **verifica con aserciones** el resultado.
4. Borra el repositorio al terminar.

Ejecuté ambos el 2026-10-15 con Git 2.55.0 en Windows 11. Los dos pasaron las 6 verificaciones; las salidas completas están en `salida-demo-bash.txt` y `salida-demo-powershell.txt`. Salida real de `./demo-cherry-pick.sh`:

```
=== 3. Mientras tanto, main recibe tres commits (solo uno es el fix de seguridad) ===
619bbf5 chore: actualización de dependencias (NO debe llegar a la feature)
53da4f6 fix(security): rechazar tokens expirados (CVE interno SEC-123)
7581f66 feat(reportes): módulo de reportes (NO debe llegar a la feature)
87bdf6b feat: autenticación inicial

=== 4. Identificar el commit del fix en main (por mensaje) ===
53da4f6 fix(security): rechazar tokens expirados (CVE interno SEC-123)

 auth.js | 4 +++-
 1 file changed, 3 insertions(+), 1 deletion(-)

=== 5. Aplicar SOLO el fix en la feature: git cherry-pick -x ===
[feature/new-login d7d32c6] fix(security): rechazar tokens expirados (CVE interno SEC-123)
 Date: Thu Oct 15 22:20:11 2026 -0500
 1 file changed, 3 insertions(+), 1 deletion(-)
* d7d32c6 fix(security): rechazar tokens expirados (CVE interno SEC-123)
* ed8c64c feat(login): opción recordar sesión
* 18a17c7 feat(login): nuevo diseño de la pantalla de login
| * 619bbf5 chore: actualización de dependencias (NO debe llegar a la feature)
| * 53da4f6 fix(security): rechazar tokens expirados (CVE interno SEC-123)
| * 7581f66 feat(reportes): módulo de reportes (NO debe llegar a la feature)
|/
* 87bdf6b feat: autenticación inicial

=== 6. Verificación ===
  [OK] la feature tiene el fix de seguridad
  [OK] la feature NO tiene reportes.js (cambio no relacionado de main)
  [OK] la feature NO tiene package.json (cambio no relacionado de main)
  [OK] el trabajo propio de la feature se conserva
  [OK] trazabilidad: el mensaje referencia el commit original (-x)

Mensaje del commit en la feature:
    fix(security): rechazar tokens expirados (CVE interno SEC-123)

    (cherry picked from commit 53da4f6b08ee3e6b34d838f1ef01f854d8e66b02)

=== 7. Más adelante: integrar la feature en main (¿se duplica el fix?) ===
git cherry main feature/new-login  ('-' = cambio equivalente ya presente en main):
    + 18a17c793bbf881041b3a2e3474bdc08f1a10a9f feat(login): nuevo diseño de la pantalla de login
    + ed8c64c3ee86219d0393d2db0eaa39c097574375 feat(login): opción recordar sesión
    - d7d32c6a8b51d6707bb0e5d03fbd693e0ff401f8 fix(security): rechazar tokens expirados (CVE interno SEC-123)
  [OK] merge sin conflictos: Git reconoce que el fix ya está aplicado
Ocurrencias de la línea del fix en auth.js de main: 1
```

(Se omitieron los pasos 1, 2 y el grafo final por brevedad; están en el archivo de salida.)

Qué demuestra: la feature recibe el commit del fix (`d7d32c6` , copia de `53da4f6`) y **no** los otros dos commits de `main` . Al integrar después la feature, `git cherry` reconoce el fix como cambio ya presente (`-`), el merge no tiene conflictos y la línea corregida aparece una sola vez.

Módulo 5: Liderazgo técnico y gestión

> **Contexto.** Equipo de 8 desarrolladores, 40 % de deuda técnica en servicios clave, 10 incidentes críticos el último mes, una funcionalidad comprometida para dentro de 3 semanas, sin estándares, *code review* irregular y dos juniors con brechas técnicas importantes.

Respuesta corta. Estabilizaría antes de acelerar. En las dos primeras semanas me concentraría en contener los incidentes, acordar con negocio un MVP que sí quepa en 3 semanas e instalar unos pocos estándares obligatorios (DoD, revisión y CI con *quality gate*). Organizaría al equipo en dos frentes con rotación y mediría el avance con métricas DORA a nivel de equipo.

1. Cinco prioridades para las primeras 2 semanas

1. **Contener los incidentes:** guardia rotativa, clasificación por severidad y un análisis de los 10 incidentes para encontrar las 2–3 causas que más se repiten.
2. **Acordar el alcance de la funcionalidad:** recortarla a un MVP con negocio, con los riesgos escritos.
3. **Diagnosticar la deuda con datos:** cruzar deuda, incidentes y frecuencia de cambio por servicio, y atacar primero donde coinciden.
4. **Instalar estándares mínimos:** DoD, revisión obligatoria con checklist y CI con *quality gate* en lo que se toque.
5. **Conocer a las personas:** 1:1 con los 8, y un plan de crecimiento con *pairing* para los dos juniors.

Días (hábiles)	Foco	Resultado al cierre del bloque
1–2	Escuchar y medir: 1:1, revisión de los incidentes, del pipeline y de cómo se despliega hoy	Línea base: incidentes por servicio y causa, métricas DORA aproximadas, mapa de riesgos
3–5	Contener y acordar: guardia rotativa, *runbooks* y alertas para las causas más frecuentes; sesión con negocio para definir el MVP	Guardia operando; MVP y riesgos acordados por escrito
6–8	Estándares: DoD, plantilla de PR con checklist, rama `main` protegida, *quality gate* en CI aplicado al código nuevo ("*clean as you code*")	Reglas activas desde ese momento; nada se fusiona sin revisión y CI en verde
9–10	Organizar y comunicar: frentes de trabajo, parejas, backlog de deuda priorizado, primera retrospectiva, informe a negocio con datos	Plan desde la semana 3 y expectativas alineadas

2. Organización del equipo

Frente	Personas	Responsabilidad
Entrega	5 (3 seniors o semi-seniors + 1 junior + 1 apoyo de QA/DevOps si existe)	MVP de la funcionalidad, detrás de *feature flag*
Estabilización	3 (1 senior + 1 semi-senior + 1 junior)	Incidentes, causas raíz y deuda de los servicios clave

- **Rotación cada sprint (2 semanas):** una o dos personas cambian de frente. Así el conocimiento se comparte y estabilizar no se vuelve "trabajo de segunda".
- **Guardia de incidentes semanal:** una persona primaria (del frente de estabilización) y un senior de respaldo. Protege la concentración del frente de entrega.
- **Juniors:**
  - *Pairing* diario con un senior, no solo revisión al final, y tareas acotadas con criterios de aceptación claros.
  - Plan de 30-60-90 días sobre sus brechas concretas (por ejemplo, pruebas, SQL o seguridad).
  - Revisiones de código con retroalimentación que explique el *por qué*.
  - Entran a la guardia acompañados después del primer mes.
- **Cadencia:** daily de 15 min, refinamiento semanal con negocio, demo y retrospectiva cada 2 semanas.

3. Métricas del área

Son **métricas de equipo para mejorar el sistema, no para evaluar personas**. Usadas individualmente se distorsionan (se optimiza el número, no el resultado). Primero se mide la línea base y después se fijan metas de tendencia.

Métrica	Qué indica	Meta inicial (a validar con la línea base)
Frecuencia de despliegue (DORA)	Capacidad de entregar en lotes pequeños	Al menos semanal por servicio

Métrica	Qué indica	Meta inicial (a validar con la línea base)
Lead time de cambios (DORA)	Tiempo de commit a producción	Reducción sostenida mes a mes
Tasa de fallos en cambios (DORA)	Calidad de lo que se despliega	< 15 %
MTTR (DORA)	Capacidad de recuperación	< 1 h para severidad 1
Incidentes críticos por mes y por servicio	Estabilidad (hoy: 10)	Tendencia a la baja; p. ej. la mitad en un trimestre
Quality gate de SonarQube (código nuevo)	Que no se agregue deuda	100 % de los PR pasan: 0 bugs o vulnerabilidades críticas, cobertura ≥ 80 %
Ratio de deuda técnica en servicios clave	Evolución del 40 %	Baja trimestre a trimestre
Predictibilidad del sprint	% de compromisos cumplidos	≥ 80 %
Tiempo hasta la primera revisión de un PR	Salud del *code review*	< 1 día hábil

4. Prácticas técnicas obligatorias

Práctica	Regla concreta
Definition of Done	Código revisado, pruebas automatizadas, *quality gate* en verde, documentación o ADR si aplica, desplegado en *staging* y con observabilidad (logs y métricas)
Code review	Al menos 1 aprobación (2 en servicios críticos), PR de menos de ~400 líneas. Checklist: funcionalidad, pruebas, seguridad OWASP (validación de entrada, autorización, secretos, sin datos personales en logs), manejo de errores y observabilidad
CI con *quality gate*	Build, pruebas, cobertura del código nuevo, análisis estático (SAST) y de dependencias. `main` protegida: no se fusiona con CI en rojo
Ramas y commits	Ramas cortas (`feature/*` , `fix/*`) sobre `main` , *Conventional Commits*, PR enlazado a la historia de usuario
Postmortems sin culpa	Para severidad 1 y 2, en menos de 5 días; las acciones tienen dueño y fecha y se siguen en el backlog
ADRs	Toda decisión con impacto (tecnología, contrato o patrón) se registra con contexto, decisión y alternativa descartada
Uso responsable de IA	Se permiten asistentes de código, pero el código generado pasa **la misma revisión y las mismas pruebas**, el autor responde por él, no se incluyen datos sensibles ni secretos en los *prompts* y solo se usan herramientas aprobadas por la empresa

5. Gestionar la presión del negocio sin comprometer la calidad

- **El alcance se negocia, la calidad no.** La fecha se cumple recortando a un **MVP** (priorización MoSCoW con negocio), no saltándose el DoD.
- **Riesgos con datos:** 10 incidentes críticos al mes tienen un costo para el negocio. Presento ese costo junto con lo que implicaría entregar sin pruebas, para que la decisión de alcance sea conjunta e informada.
- ***Feature flags*:** desplegar temprano y oculto, activar gradualmente y apagar sin redesplegar si algo falla.
- **Transparencia semanal:** un semáforo de alcance, fecha y riesgos, avisando temprano y no el día anterior.
- **Capacidad fija para deuda:** ~20 % de cada sprint, que no se renegocia sprint a sprint, ligada a la reducción de incidentes medida en §3.

6. Evidencia de estas prácticas en esta prueba

Desarrollé esta prueba con un asistente de IA y revisé y verifiqué cada entregable. Varias de las prácticas que propongo las apliqué aquí, a escala de una persona:

Práctica propuesta	Cómo la apliqué en la prueba
CI con *quality gate*	JaCoCo hace fallar el build del backend si la cobertura de líneas baja del 80 % (96 pruebas)
ADRs	`DECISIONES.md` en backend y frontend: cada decisión con su alternativa descartada y los supuestos tomados
Pruebas más allá de las unitarias	Las pruebas e2e con Playwright encontraron **3 defectos reales** que las unitarias no detectaban (recarga tras un 409, un 403 de CORS detrás de nginx y la zona horaria del contenedor)
IA con revisión humana	Todo lo generado se verificó con build, pruebas, revisiones visuales en el navegador y mediciones reales (por ejemplo, los planes de ejecución del Módulo 3)
Honestidad sobre el estado	El diseño (Módulo 1) distingue en una tabla lo implementado, lo simplificado y lo propuesto